МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
“Национальный исследовательский университет ИТМО”

**ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ**

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине
“Бизнес-логика программных систем”

Вариант: 555.

Работу выполнили:
Студенты потока 1.5
Коломиец Никита Сергеевич
Болорболд Аригуун
Лектор:
Цопа Евгений Алексеевич
Практик:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург
2025 г.

Содержимое

1 Текст задания **3**
1.1 Порядок выполнения работы 3

2 Выполнение **3**
2.1 BPMN 3
2.2 Программная реализация 5
2.3 UML-диаграммы 9

3 Реализация конфигурации для тестирования **10**

4 Запуск на сервере helios **10**

5 Вывод **11**

1 Текст задания

Описать бизнес-процесс в соответствии с нотацией BPMN 2.0, после чего реализовать его в виде приложения на базе Spring Boot.

Вариант №555: Google Play — приложения и игры — https://play.google.com. Бизнес-процесс: взаимодействие клиента с каталогом — выбор приложения, установка и/или покупка, управление картами и платёжными аккаунтами.

1.1 Порядок выполнения работы

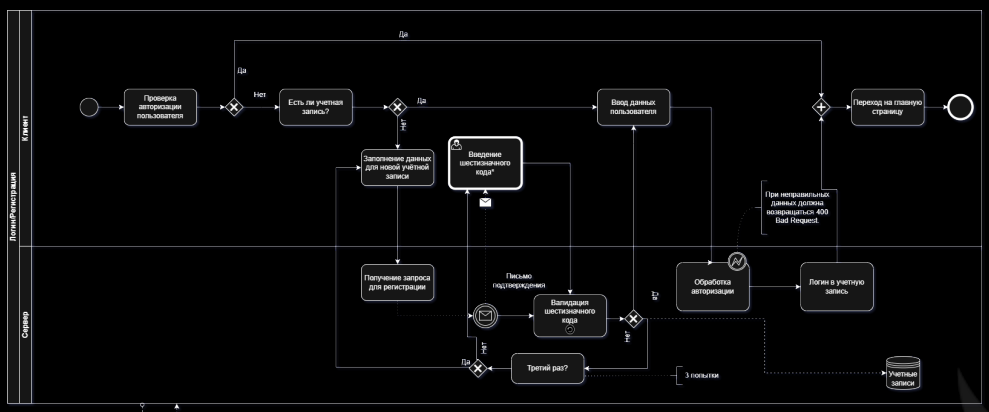
- Выбрать один из бизнес-процессов, реализуемых сайтом из варианта задания.
- Утвердить выбранный бизнес-процесс у преподавателя.
- Специфицировать модель реализуемого бизнес-процесса в соответствии с требованиями BPMN 2.0.
- Разработать приложение на базе Spring Boot, реализующее описанный на предыдущем шаге бизнес-процесс. Приложение должно использовать СУБД PostgreSQL для хранения данных, для всех публичных интерфейсов должны быть разработаны REST API.
- Разработать набор curl-скриптов, либо набор запросов для REST клиента Insomnia для тестирования публичных интерфейсов разработанного программного модуля. Запросы Insomnia оформить в виде файла экспорта.
- Развернуть разработанное приложение на сервере `helios`.

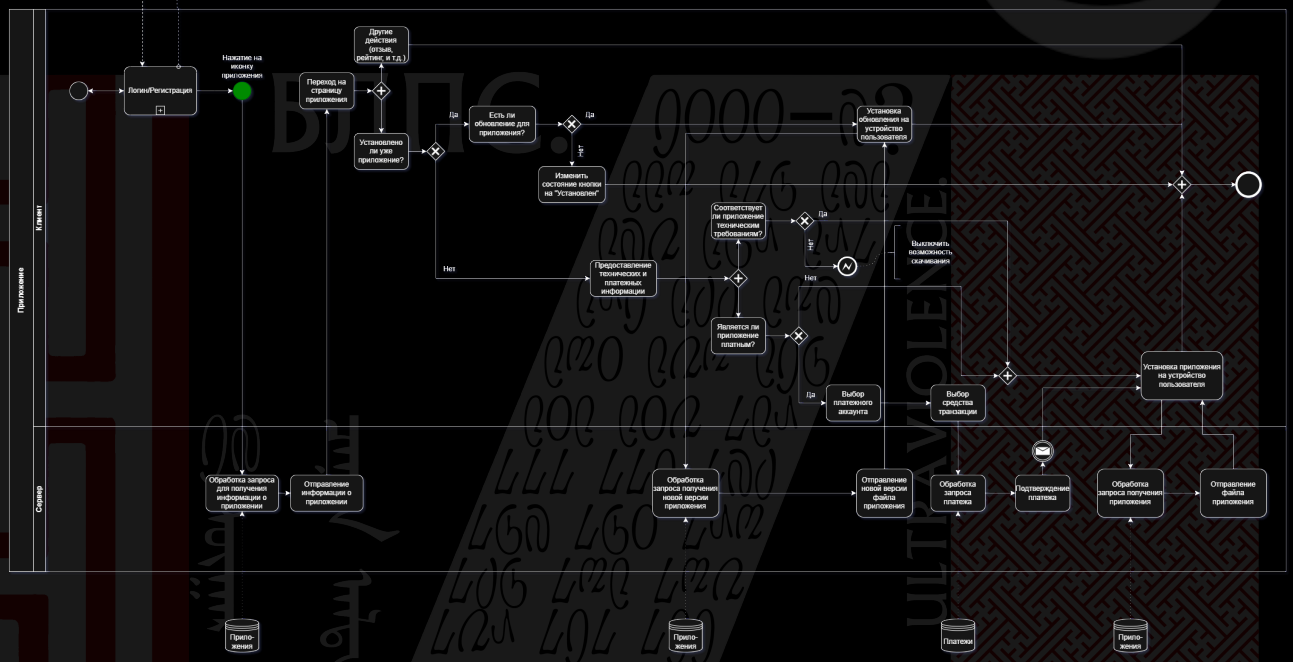
2 Выполнение

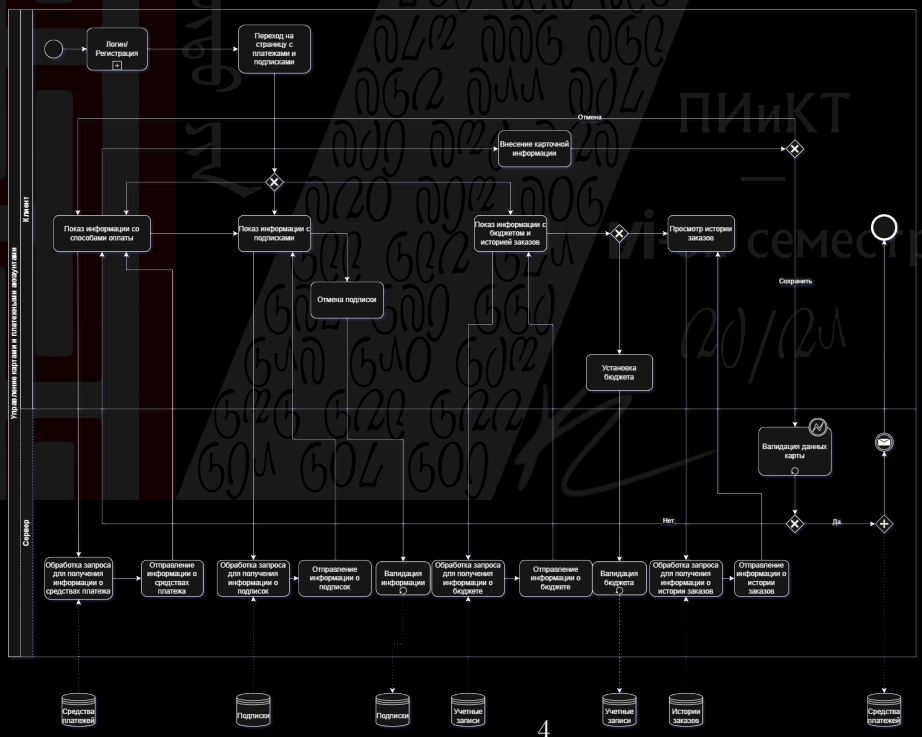
Ссылка на репозиторию: GitHub
XVIIStarPlatinum — Аригуун;
nkolomiika — Никита;

2.1 BPMN

Спецификация бизнес-процессов взаимодействия клиента с каталогом — выбор приложения, установка и/или покупка, управление картами и платёжными аккаунтами.


ЛогинРегистрация
Клиент
Сервер
Проверка авторизации пользователя
Есть ли учетная запись?
Да
Нет
Заполнение данных для новой учетной записи
Введение шестизначного кода*
Ввод данных пользователя
Переход на главную страницу
При неправильных данных должна возвращаться 400 Bad Request
Получение запроса для регистрации
Письмо подтверждения
Валидация шестизначного кода
Третий раз?
3 попытки
Обработка авторизации
Логин в учетную запись
Учетные записи


Приложение
Клиент
Сервер
Логин/Регистрация
Нажатие на иконку приложения
Переход на страницу приложения
Другие действия (отзыв, рейтинг и т.д.)
Установлено ли уже приложение?
Есть ли обновление для приложения?
Да
Нет
Изменить состояние кнопки на "Установлено"
Установка обновления на устройство пользователя
Предоставление технических и платежных информации
Соответствует ли приложение техническим требованиям?
Выключить возможность скачивания
Является ли приложение платным?
Выбор платежного аккаунта
Выбор средства транзакции
Установка приложения на устройство пользователя
Обработка запроса для получения информации о приложении
Отправление информации о приложении
Обработка запроса получения новой версии приложения
Отправление новой версии файла приложения
Обработка запроса платежа
Подтверждение платежа
Обработка запроса получения приложения
Отправление файла приложения
Приложения
Платежи


Управление картами и платежными аккаунтами
Клиент
Сервер
Логин/ Регистрация
Переход на страницу с платежами и подписками
Отмена
Внесение карточной информации
Показ информации со способами оплаты
Показ информации с подписками
Показ информации с бюджетом и историей заказов
Просмотр истории заказов
Сохранить
Отмена подписки
Установка бюджета
Валидация данных карты
Нет
Да
Обработка запроса для получения информации о средствах платежа
Отправление информации о средствах платежа
Обработка запроса для получения информации о подписок
Отправление информации о подписок
Валидация информации
Обработка запроса для получения информации о бюджете
Отправление информации о бюджете
Валидация бюджета
Обработка запроса для получения информации о истории заказов
Отправление информации о истории заказов
Средства платежей
Подписки
Учетные записи
История заказов

2.2 Программная реализация

Для сущности уровня БД были созданы классы. Ниже — один пример такого класса:

```
package com.example.backend.model.data.app;

import com.example.backend.model.auth.User;
import com.example.backend.model.data.Review;
import com.example.backend.model.data.subscriptions.Subscription;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.UuidGenerator;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.UUID;

@Entity
@Table(name = "apps")
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class App {

    @Id
    @UuidGenerator
    private UUID id;

    @NotBlank
    @Column(length = 32, nullable = false)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String description;

    @NotNull
    @PositiveOrZero
    private Double price;

    @NotNull
    private LocalDate releaseDate;

    @ManyToOne
    @JoinColumn(name = "author_id", nullable = false)
    private User author;

    @Builder.Default
```

```
    @OneToMany(mappedBy = "app", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Subscription> subscriptions = new ArrayList<>();

    @JsonIgnore
    @Builder.Default
    @OneToMany(mappedBy = "app", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    @JsonIgnore
    @ManyToMany(mappedBy = "downloadedApps")
    private Set<User> usersWhoDownloaded;

    @JsonIgnore
    @Builder.Default
    @OneToMany(mappedBy = "app", cascade = CascadeType.ALL, orphanRemoval = true)
    List<AppVersion> appVersions = new ArrayList<>();

    @JsonIgnore
    @NotNull
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    private AppFile appFile;

    public boolean hasSubscriptions() {
        return !subscriptions.isEmpty();
    }

    public boolean isNewerThan(App app) {
        return app.getId().equals(this.id) && app.appVersions.size() < this.appVersions.size();
    }

    public AppVersion getLatestVersion() {
        return !appVersions.isEmpty() ? appVersions.get(appVersions.size() - 1) : null;
    }
}
```

Контроллер для взаимодействия с приложениями через API использует следующие эндпоинты:

```
GET: /api/v1/apps
GET: /api/v1/apps/{appId}
GET: /api/v1/apps/{appId}/compatibility
GET: /api/v1/apps/{appId}/download-info
GET: /api/v1/apps/{appId}/reviews
GET: /api/v1/apps/{appId}/reviews/average
GET: /api/v1/apps/download
POST: /api/v1/apps
POST: /api/v1/apps/{appId}/reviews
DELETE: /api/v1/apps/{appId}
```

Для сообщения с пользовательской частью предоставлены следующие интерфейсы класса `App` — `AppCreateRequest`, `AppDownloadResponse`, `AppRequirementsDto`, `AppUpdateDto`. Бизнес-логика описана в классе `AppService` с необходимыми зависимостями и вспомогательными сервисами — сервисы, к которым напрямую не связаны контроллеры. Помимо этого, настроен доступ к эндпоинтам по ролям с помощью Spring Security.

Ниже приведен фрагмент класса `AppController`, метод getApps:

```
@RestController
@RequestMapping("/api/v1/apps")
@RequiredArgsConstructor
```

```
public class AppController {
    private final AppService appService;
    private final ReviewService reviewService;

    @GetMapping
    public ResponseEntity<List<App>> getApps() {
        return ResponseEntity.ok(appService.getAllAvailableApps());
    }
    ...
}
```

В запросах POST и PUT в теле запроса ожидается объект класса `AppCreateDto`:

```
    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @PreAuthorize("hasRole('DEVELOPER') || hasRole('MODERATOR') || hasRole('ADMIN')")
    public ResponseEntity<App> createApp(@Valid @ModelAttribute AppCreateRequest appCreateDto,
                                         @RequestParam("file") MultipartFile file) {
        return ResponseEntity.status(HttpStatus.CREATED).body(appService.createApp(appCreateDto, file
    }
```

Так как бизнес-логика маркетплейса подразумевает управление картами, была создана сущность уровня БД `Card`:

```
package com.example.backend.model.data.finances;

import com.example.backend.model.auth.User;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.UuidGenerator;

import java.time.LocalDate;
import java.util.UUID;

@Entity
@Table(name = "cards")
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class Card {
    @Id
    @UuidGenerator
    private UUID id;

    @ManyToOne(optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    @JsonIgnore
    private User user;

    @NotNull
    @Size(min = 16, max = 19)
```

```
    @Pattern(regexp = "\\d+")
    private String number;

    @NotNull
    @Pattern(regexp = "^\\d{3}$")
    private String cvv;

    @NotNull
    private LocalDate expiryDate;

    @NotNull
    @Builder.Default
    private double balance = 0;

    @NotNull
    @Builder.Default
    private Boolean isDefault = false;
}
```

Для сообщения с пользовательской частью предоставлены следующие интерфейсы класса `Card` — `CardDto` и `DepositRequest`. Бизнес-логика описана в классе `CardService` с необходимыми зависимостями и вспомогательными сервисами. Настроен доступ к эндпоинтам через аннотацию `@PreAuthorize("isAuthenticated()")` в связи с установленной бизнес-логикой.

Контроллер для взаимодействия с приложениями через API использует следующие эндпоинты:

```
GET: /api/v1/cards
POST: /api/v1/cards
PUT: /api/v1/cards/{cardId}
PUT: /api/v1/cards/deposit
DELETE: /api/v1/cards/{cardId}
```

В классе CardService (бизнес-логика обработки операций с картами) использует аннотацию `@Transactional` в связи с необходимостью отражать изменение состояния карты только на удачно добавленный в базу данных картов.

Работа с базой данных реализуется с помощью `JPA` и `Hibernate` и настраивается в файле `application.properties`.

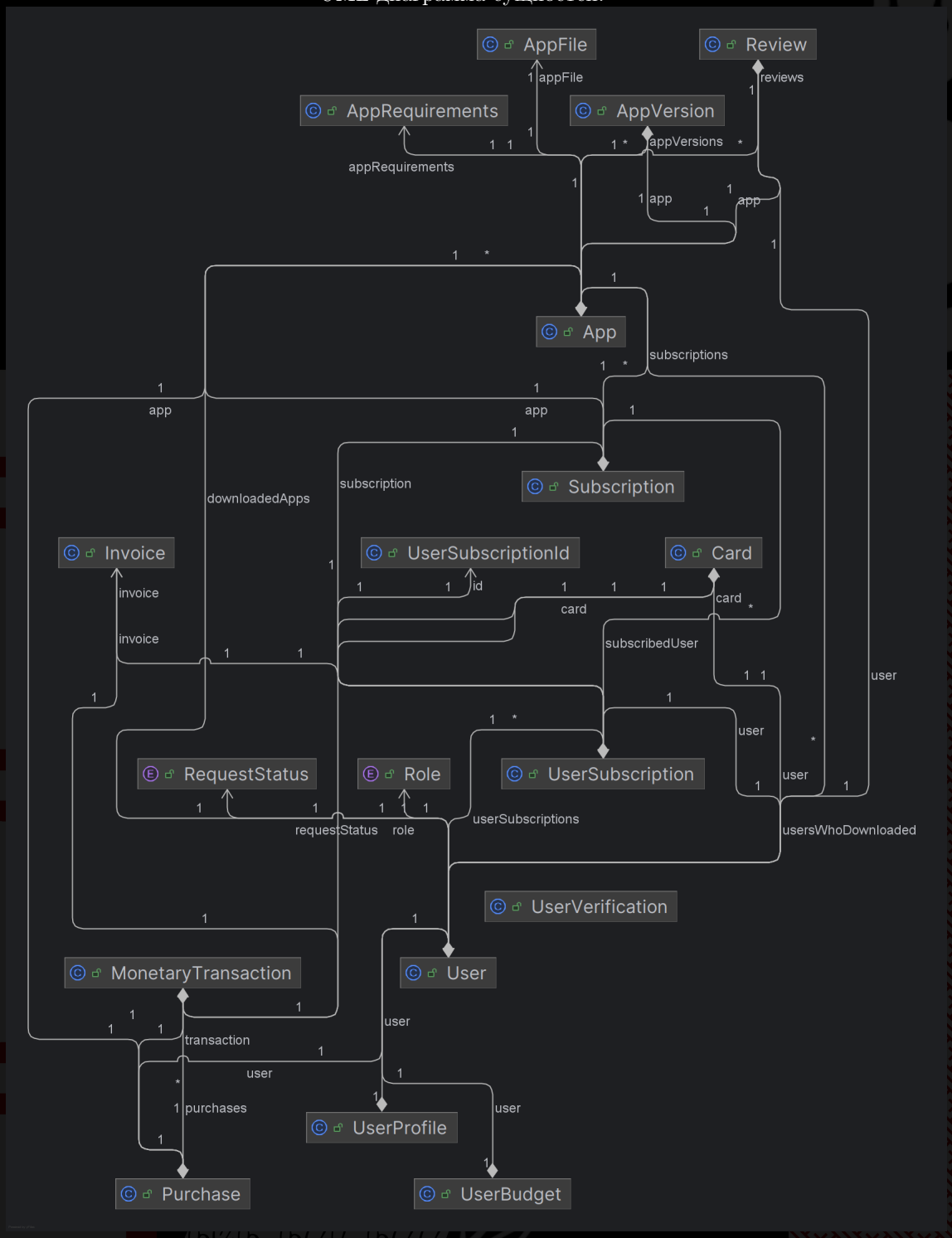
Сборка приложения происходит с помощью maven, поэтому подключение зависимостей описывается в `pom.xml`:

```
    ...
    <dependency>
        <groupId>jakarta.validation</groupId>
        <artifactId>jakarta.validation-api</artifactId>
        <version>3.1.1</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
    </dependency>
    ...
```

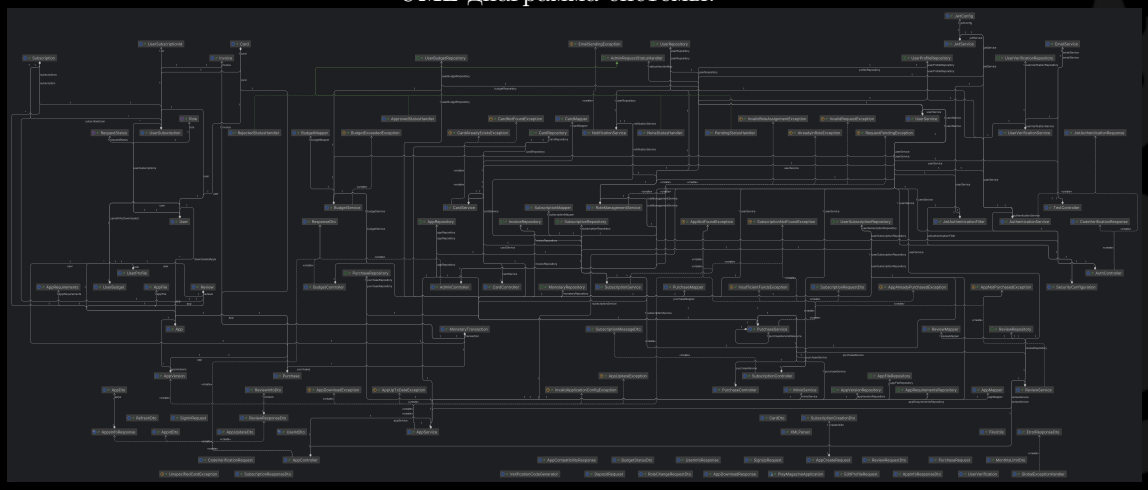
Здесь частично объясняется только 2 из 6 (без учёта `User`) возможных наборов действии. Для дальнейшего чтения обратитесь в GitHub-страницу, где в формате `Markdown` всё расписано подробно.

2.3 UML-диаграммы

UML-диаграмма сущностей:



UML-диаграмма системы:



В более хорошем качестве можно увидеть отсюда.

3 Реализация конфигурации для тестирования

В связи с использованием Spring Security и JWT (**а также 2FA (!)**), с помощью которого происходило сообщение авторизированного пользователя с приложением, необходимо хранить токен авторизации. Например, получение токена для тестового юзера с логином 1 и паролем 1:

Ниже приведена конфигурация для тестирования REST API взаимодействия пользователя с приложениями:

Объект комментария представлен в формате JSON, передаётся с помощью флага -d, а токен передаётся посредством Cookie. Тестовый объект записан непосредственно в параметрах curl.

Порт изменён со стандартного на 18018 во избежание конфликтов при запуске на сервере `helios`.

Кроме curl-скриптов для управления приложениями, созданы конфигурации для взаимодействия с картами:

4 Запуск на сервере helios

Шаги:

1. Подготовили application.properties для работы на удаленном сервере (изменили подключение к БД и указали порт сервера):

```
spring.datasource.url=jdbc:postgresql://pg:5432/studs
spring.datasource.username=s******
spring.datasource.password= ‘‘~/pgpass’’
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
server.port=18018
```

2. Собрали исполняемый файл при помощи `mvn clean package -DskipTests`;
3. Полученный артефакт перенесли на helios;
4. Переподключились к helios следующим образом: `ssh -p 2222 s******@helios.cs.ifmo.ru -L 8080:helios.c`
5. Запустили .jar файл

6. Перешли при помощи `psql -h pg -d studs` в БД и заполнили БД тестовыми данными через конфигурации на `Insomnia`;
7. Теперь можно взаимодействовать с API по адресу localhost:8080.

5 Вывод

В процессе выполнения лабораторной работы мы научились многому. Это включает, но не ограничивается:

- усовершенствованию CDI и IoC;
- улучшенная логика при обработке исключении;
- составление бизнес-логики по BPMN 2.0;
- составление тестовых сценарий через `Insomnia`;
- двухфакторная аутентификация;
- политика доступа (ролевая система);

Это было действительно интересным опытом, особенно учитывая того, что один из нас (бонус баллы за правильный ответ) решил сильно усложнить всю систему, когда можно было обойтись принципиально просто (например, только взаимодействием с комментариями, если вы знаете о чём это), так как предмет не предназначен быть сложным и ориентирован на изучение некоторых практик бизнес-логики архитектуры `MVC`. Так что это станет для нас наиценным опытом.